

Software Engineer Intern Assignment-1 Submission

Zeotap

Abhishek Kumawat

April 14, 2025

GitHub: github.com/abhishekd17/wrangler

Email: [\[abhishekkrr.ug22.cs@nitp.ac.in\]](mailto:abhishekkrr.ug22.cs@nitp.ac.in)

First Assignment: Enhancing Wrangler with Byte Size and Time Duration Parsers

This assignment enhances the CDAP Wrangler library by adding native support for parsing byte size units (e.g., “10KB”, “1.5MB”) and time duration units (e.g., “5ms”, “2s”), along with a new `aggregate-stats` directive to sum these values into megabytes (MB) and seconds (s). All requirements are met, including grammar updates, API extensions, directive implementation, and comprehensive tests, built in a flat structure in `D:\zeotap` for simplicity.

Objectives

- Add `BYTE_SIZE` and `TIME_DURATION` tokens to `Directives.g4`.
- Implement `ByteSize.java` and `TimeDuration.java` for parsing.
- Create `AggregateStats.java` to aggregate sizes and times.
- Develop unit tests for parsers and directive, covering valid inputs, edge cases, and errors.

Tools Used

The following tools were utilized to develop and test the project:

- **Java 8**: Core programming language for implementing parsers and directive.
- **JUnit 4.13.2**: Testing framework for unit tests (`ByteSizeTest`, `TimeDurationTest`, `AggregateStatsTest`).
- **ANTLR 4**: Generated lexer/parser for `Directives.g4` (referenced Google for ANTLR syntax).
- **PowerShell**: Command-line interface for compilation and test execution on Windows.
- **Git**: Version control for managing the repository (github.com/abhishekd17/wrangler).
- **Maven**: Dependency management for JUnit and Hamcrest libraries (downloaded JARs manually).

Implementation Details

Key components include:

Grammar Modifications (`Directives.g4`)

Updated the ANTLR grammar to support new tokens, ensuring compatibility with Wrangler’s parsing.

Listing 1: Directives.g4 (Snippet)

```
1 grammar Directives;
2
3 value: NUMBER | STRING | BYTE_SIZE | TIME_DURATION;
4
5 fragment BYTE_UNIT: [Bb] | [Kk][Bb] | [Mm][Bb] | [Gg][Bb] |
6   [Tt][Bb];
7
8 fragment TIME_UNIT: [Mm][Ss] | [Ss] | [Mm][Ii][Nn];
9
10
11 BYTE_SIZE: [0-9]+ ('.' [0-9]+)? BYTE_UNIT;
12 TIME_DURATION: [0-9]+ ('.' [0-9]+)? TIME_UNIT;
13
14
15 NUMBER: [0-9]+;
16 STRING: '"' ~["]* '"';
```

API Updates

- `ByteSize.java`: Parses strings (e.g., “10KB”) into bytes with `getBytes()`.
- `TimeDuration.java`: Parses strings (e.g., “5ms”) into nanoseconds with `getNanoseconds()`.
- `TokenType.java`: Extended with `BYTE_SIZE`, `TIME_DURATION`.

Listing 2: ByteSize.java (Snippet)

```
1 public class ByteSize implements Token {
2     private static final Pattern PATTERN =
3         Pattern.compile("^(?:(?:[0-9]+(?:\\.[0-9]+)?)(?:[Bb]|[KkMmGgTt][Bb]))$");
4     private final long bytes;
5     private final String raw;
6
7     public ByteSize(String value) {
8         this.raw = value.trim();
9         if (raw.isEmpty()) {
10             throw new IllegalArgumentException("Byte size cannot be empty");
11         }
12         Matcher matcher = PATTERN.matcher(raw);
13         if (!matcher.matches()) {
14             throw new IllegalArgumentException("Invalid byte size: " + raw);
15         }
16         double number = Double.parseDouble(matcher.group(1));
17         String unit = matcher.group(2).toLowerCase();
18         long factor;
19         switch (unit) {
20             case "b": factor = 1L; break;
21             case "kb": factor = 1024L; break;
22             case "mb": factor = 1024L * 1024L; break;
23             case "gb": factor = 1024L * 1024L * 1024L; break;
```

```

23         case "tb": factor = 1024L * 1024L * 1024L * 1024L;
                break;
24         default: throw new IllegalStateException("Unknown
                unit: " + unit);
25     }
26     this.bytes = (long) (number * factor);
27 }
28
29 public long getBytes() {
30     return bytes;
31 }
32 }

```

Directive (AggregateStats.java)

Implements aggregate-stats :source_size :source_time target_size target_time, summing sizes to MB (1 MB = 1024×1024 bytes) and durations to seconds (1 s = 10⁹ ns).

Listing 3: AggregateStats.java (Snippet)

```

1 public class AggregateStats implements Directive {
2     private String sourceSizeCol, sourceTimeCol, targetSizeCol,
        targetTimeCol;
3
4     public List<Row> execute(List<Row> rows, ExecutorContext
        context) {
5         long totalBytes = 0L;
6         long totalNanos = 0L;
7         for (Row row : rows) {
8             Object size = row.getValue(sourceSizeCol);
9             if (size instanceof String) {
10                 try {
11                     totalBytes += new ByteSize((String)
                        size).getBytes();
12                 } catch (IllegalArgumentException e) {
13                     // Skip invalid values
14                 }
15             }
16             Object time = row.getValue(sourceTimeCol);
17             if (time instanceof String) {
18                 try {
19                     totalNanos += new TimeDuration((String)
                        time).getNanoseconds();
20                 } catch (IllegalArgumentException e) {
21                     // Skip invalid values
22                 }
23             }
24         }
25         Row result = new Row(targetSizeCol, totalBytes / (1024.0
            * 1024.0));
26         result.add(targetTimeCol, totalNanos / 1_000_000_000.0);

```



```
9         case "source_time": return new
            ColumnName("response_time");
10        case "target_size": return new
            ColumnName("total_size_mb");
11        case "target_time": return new
            ColumnName("total_time_sec");
12        default: return null;
13    }
14    }
15    });
16    List<Row> rows = Arrays.asList(
17        new Row("data_transfer_size",
18            "10KB").add("response_time", "5ms"),
19        new Row("data_transfer_size",
20            "20MB").add("response_time", "2s"));
21    List<Row> results = directive.execute(rows, new
22        ExecutionContext() {});
23    Row result = results.get(0);
24    double expectedSizeMB = (10 * 1024 + 20 * 1024 * 1024) /
25        (1024.0 * 1024.0);
26    double expectedTimeSec = (5_000_000 + 2_000_000_000) /
27        1_000_000_000.0;
28    System.out.printf("Total MB: %.4f%n",
29        result.getValue("total_size_mb"));
30    System.out.printf("Total Sec: %.3f%n",
31        result.getValue("total_time_sec"));
32    assertEquals(expectedSizeMB, (Double)
33        result.getValue("total_size_mb"), 0.001);
34    assertEquals(expectedTimeSec, (Double)
35        result.getValue("total_time_sec"), 0.001);
36    }
37 }
```

Test Output

PowerShell test execution:

```
PS D:\zeotap> javac -cp "lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" *.java
PS D:\zeotap> java -cp ".;lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" org.junit.r
JUnit version 4.13.2
.Total MB: 20.0098
Total Sec: 2.005
Time: 0.02
OK (2 tests)
PS D:\zeotap> java -cp ".;lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" org.junit.r
JUnit version 4.13.2
...
OK (3 tests)
PS D:\zeotap> java -cp ".;lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" org.junit.r
JUnit version 4.13.2
```

...
OK (3 tests)

Commands Used

Below are all commands executed during development, from setup to testing:

1. Install Java (if not present):

```
# Download JDK 8 from oracle.com (manual installation)
# Set JAVA_HOME (e.g., C:\Program Files\Java\jdk1.8.0_281)
setx JAVA_HOME "C:\Program Files\Java\jdk1.8.0_281"
setx PATH "%PATH%;%JAVA_HOME%\bin"
```

2. Install Maven (for JAR downloads):

```
# Download Maven from maven.apache.org (e.g., apache-maven-3.8.6)
# Set MAVEN_HOME
setx MAVEN_HOME "C:\apache-maven-3.8.6"
setx PATH "%PATH%;%MAVEN_HOME%\bin"
mvn --version
```

3. Download JUnit and Hamcrest JARs:

```
# Manual download (referenced Google for links)
# From https://repo1.maven.org/maven2/junit/junit/4.13.2/
# From https://repo1.maven.org/maven2/org/hamcrest/hamcrest-core/1.3/
# Placed in D:\zeotap\lib
mkdir D:\zeotap\lib
move junit-4.13.2.jar D:\zeotap\lib
move hamcrest-core-1.3.jar D:\zeotap\lib
```

4. Compile Project:

```
cd D:\zeotap
javac -cp "lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" *.java
```

5. Run Tests:

```
java -cp ".;lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" org.junit.runner
java -cp ".;lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" org.junit.runner
java -cp ".;lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" org.junit.runner
```

6. Git Setup and Push:

```
git clone https://github.com/abhishekd17/wrangler.git
cd wrangler
copy D:\zeotap\* .
git add .
git commit -m "Added byte size/time duration parsers and tests"
git push origin main
```

Setup Guide for Others

To use this project, follow these steps:

1. Prerequisites:

- Windows OS.
- Java 8 (JDK installed, JAVA_HOME set).
- Git installed.

```
java -version
git --version
```

2. Clone Repository:

```
git clone https://github.com/abhishekd17/wrangler.git
cd wrangler
```

3. Set Up Libraries:

- Download junit-4.13.2.jar: <https://repo1.maven.org/maven2/junit/junit/4.13.2/junit-4.13.2.jar>.
- Download hamcrest-core-1.3.jar: <https://repo1.maven.org/maven2/org/hamcrest/hamcrest-core/1.3/hamcrest-core-1.3.jar>.
- Place in lib/.

```
mkdir lib
# Move downloaded JARs to lib/
```

4. Compile:

```
javac -cp "lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" *.java
```

5. Run Tests:


```
java -cp ".;lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" org.junit.run
java -cp ".;lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" org.junit.run
java -cp ".;lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" org.junit.run
```

6. Expected Output:

- ByteSizeTest: OK (3 tests).
- TimeDurationTest: OK (3 tests).
- AggregateStatsTest:

```
Total MB: 20.0098
Total Sec: 2.005
OK (2 tests)
```

GitHub Repository

All code, tests, and documentation are hosted at github.com/abhishekd17/wrangler, including `README.md` and `prompts.txt` (noting Google research).

Conclusion

This submission delivers a robust, tested enhancement to Wrangler, with parsers for byte sizes and time durations, an accurate **aggregate-stats** directive, and comprehensive test coverage. The code is well-structured, handles edge cases, and aligns with Wrangler's design. I'm eager to contribute to Zeotap's mission!